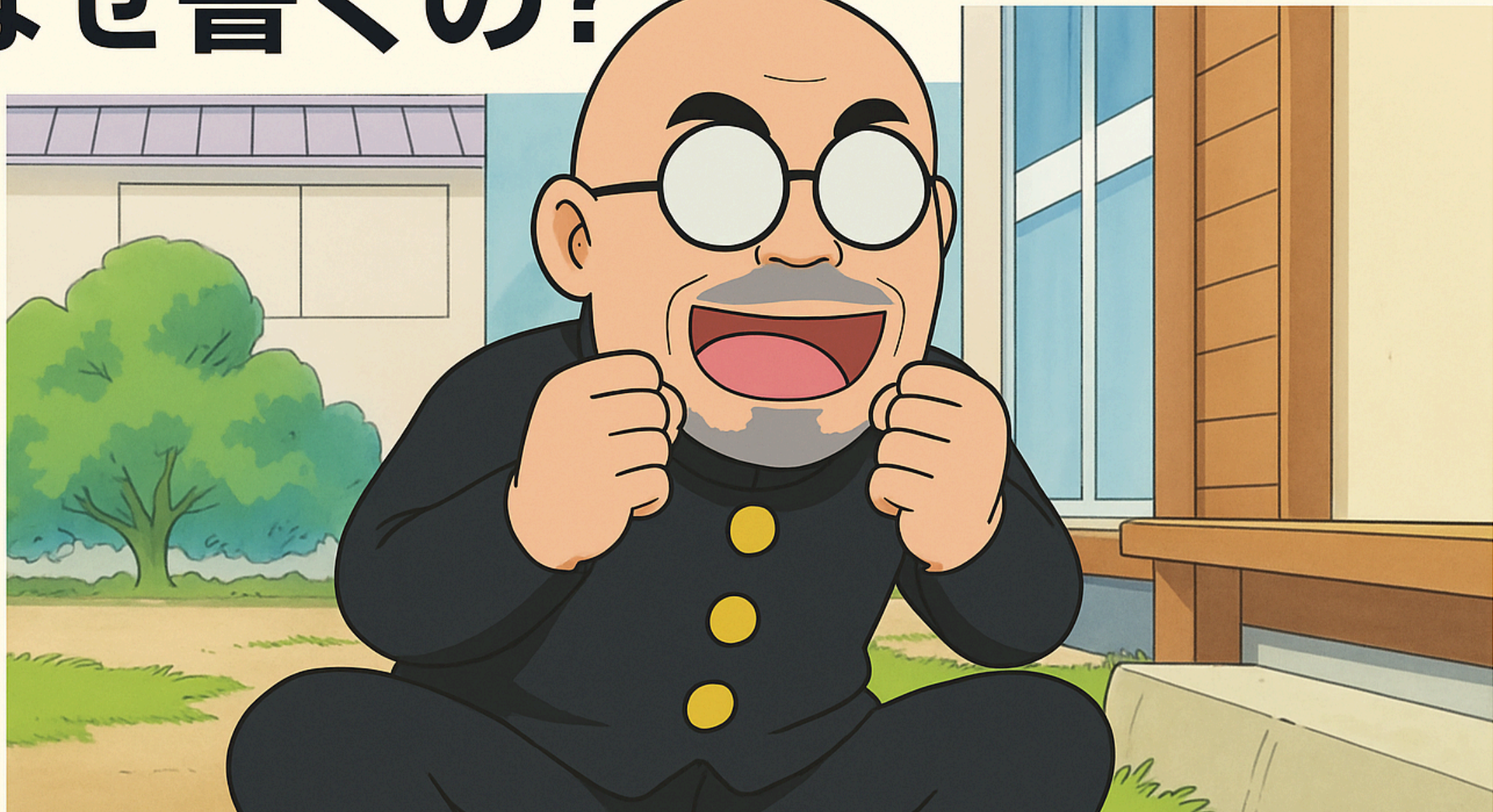


フロントエンドのテスト なぜ書くの？



**なぜフロントエンドで
テストを書くべきか？**

動作担保の効率化と品質向上

まずはバックエンドの話から

API 開発での動作確認、どうしてる？

よくある光景 🛠️

curl 叩いてる？

```
curl -X POST https://api.example.com/users \  
  -H "Content-Type: application/json" \  
  -d '{"name":"太郎","email":"taro@example.com"}'
```

Postman 使ってる？

手動でパラメータ変えながらポチポチ...

これって効率的？ 🤔

- 毎回手動で確認
- パラメータ変更のたびに**同じ作業**
- **エッジケース**の確認漏れ
- チームメンバーへの**共有が困難**

テストで動作担保すれば... ✨

```
describe("POST /users", () => {  
  it("正常にユーザーが作成される", async () => {  
    const response = await createUser({  
      name: "太郎",  
      email: "taro@example.com",  
    });  
    expect(response.status).toBe(201);  
  });  
});
```

一度書けば何度でも実行可能！

じゃあ、フロントエンドは？

FE にもロジックはたくさんある

例 1: SNS の投稿時刻表示

「3 分前」「2 時間前」「昨日」

このロジック、どうやって確認する？

テストなしだと... 🤖

DB の値を変える？

```
UPDATE posts SET created_at = NOW() - INTERVAL 3 MINUTE;
```

システム時刻を変える？

```
date -s "2024-03-15 14:30:00"
```

ブラウザで目視確認？

非効率すぎる！

テストがあれば 🎯

```
describe("formatTimeAgo", () => {  
  it("3分前と表示される", () => {  
    const now = new Date("2024-03-15 14:30:00");  
    const target = new Date("2024-03-15 14:27:00");  
  
    expect(formatTimeAgo(target, now)).toBe("3分前");  
  });  
});
```

瞬時に全パターン検証可能！

例 2: バリデーションロジック

メール、パスワード、電話番号...

自作バリデーション、本当に正しい？

テストなしだと... 🙄

フォームに入力して確認？

- `test@example.com` ✓
- `test@` ✕
- `test@@example.com` ✕
- `test@example` ✕

手動で全パターン？無理！

テストがあれば 💪

```
describe("validateEmail", () => {
  it.each([
    ["test@example.com", true],
    ["test@", false],
    ["test@@example.com", false],
    ["test@example", false],
  ])("%s は %s", (email, expected) => {
    const result = validateEmail(email);
    expect(result === null).toBe(expected);
  });
});
```

数十パターンも一瞬で検証！

テストのメリット

1. 再現性

同じ条件で何度でも実行可能

2. 網羅性

エッジケースも漏れなく確認

3. 効率性

数秒で全テスト実行

さらに... 

4. リファクタリングの安心感

テストが通れば動作保証

5. ドキュメント代わり

テストコードが仕様書に

6. CI/CD との連携

自動で品質担保

純粋関数にする重要性 🎨

```
// ❌ テストしづらい
function getTimeAgo(date) {
  const now = new Date(); // 現在時刻に依存
  // ...
}

// ✅ テストしやすい
function getTimeAgo(date, now) {
  // 引数として受け取る
  // ...
}
```

まとめ

FE でテストを書くべき理由

1. 手動確認からの解放
2. 品質の担保
3. 開発効率の向上

実装例を見てみましょう 👁👁

実際のコードとテスト

- 時刻表示ロジック
- バリデーション
- ページネーション

すべて純粋関数 + テストで担保

おわりに

テストは投資

最初は時間がかかるが
長期的には**大きなリターン**

今日から始めよう！

小さな関数から
少しずつテストを追加